

Full-module review: Mqtt_Json_Format_V3.0_Dual_Broker

Date: 03 October 2026 | Repository: REConnect_Firmware_Modules, commit 04a573a | Follows:
Code_Review_2026-09-29.pdf and Mqtt_Dual_Broker_Code_Review_2026-10-02.pdf

Scope

The 02 Oct re-review looked only at the changed code. This review covers the whole module: mqtt_connect.c, main.c, set_cfg_request.c, get_cfg_request.c, file_gen_main.c, modbus_json_export.c, health_status.c, health_check.c, cmd_resp.c, dbg_log.c, redis_hash_generator.c, the four dlms_cdf_*.c generators, the headers and the Makefile. Third-party code (cJSON, sqlite3) was not reviewed. Where useful, Redis field names were cross-checked against the modules that read them (Modbus masters, IEC 101/104, dcu_mon_proc, Modem_PPP_IPSEC). Nothing was compiled or run; all findings come from reading the source. Line numbers refer to commit 04a573a.

Status of earlier findings

The module source is **unchanged since the 02 Oct re-review** (same code, same line numbers). The 6 open findings from 29 Sep (1, 2, 3, 7, 10, 11) and all 22 findings from 02 Oct (N1-N22) are **still open**. Their details are in the earlier reports and are not repeated here. Everything below is new.

Key takeaways

- **A second root command-injection path**: modem fields from set-config end up in a shell script the DCU executes as root (F1). Together with N1 (IPsec), all set-config strings that reach scripts need validation.
- **Several remote crashes** from a single server command or a missing Redis field: F3 (METER_NAME stack overflow), F4 (get-config DLMS_ETHERNET), F9 and F10 (missing fields crash every cycle, so the process keeps restarting).
- **Thread safety**: logging from Paho callback threads uses the shared Redis context and log file without locks (F5), which undoes the earlier callback fix.
- **Data correctness**: global on-demand flags leak into cyclic data (F6, F7), meter lookups read only the first HSCAN page so meters go missing at random (F8), and generator failures lead to stale or wrong files being sent (F2).

Verification: Each of the High findings (F1-F10) was checked against the source a second time. Each finding states whether it was traced in code (Confirmed) or depends on something not visible in this repository (Plausible).

New findings - summary

#	Severity	Location	Issue
F1	High	set_cfg_request.c:288-291; Modem_PPP_IPS EC/ppp_monitor.sh:530-545, 626-651, 682	Shell and pppd option injection through MODEM fields
F2	High	file_gen_main.c:2103-2106, 2196-2214 (same pattern 2259/2277, 2308/2318, 2345/2353, 2379/2387)	Uninitialised file names passed to system("rm ...") and fopen
F3	High	set_cfg_request.c:847 (MODBUS_TCP), :940 (MODBUS_RTU)	Stack buffer overflow from METER_NAME
F4	High	get_cfg_request.c:1626-1630; mqtt_connect.c:3963-3968	Get-config DLMS_ETHERNET returns (char*)-1 and crashes
F5	High	dbg_log.c:64-96, 103-136, 291-333; main.c:117-120, 1472; mqtt_connect.c:914, 1059-1067, 1323-1329, 4145	Logging from Paho threads uses the shared Redis context and log file without locking
F6	High	mqtt_connect.c:2798/2855, 2887/2901, 2961/2982, 2990/3005; dlms_cdf_ls.c:244; dlms_cdf_event.c:106, 209-214; dlms_cdf_mn.c:36; dlms_cdf_billing.c:42-50, 196-200; file_gen_main.c:854	Global OD flags stay set and redirect cyclic data to the OD tables
F7	High	mqtt_connect.c:3896-3897, 2838-2860, 3749; file_gen_main.c:863-871	check_redis_resp has no timeout and puts the FetchDay SEQ_NUM on cyclic messages
F8	High	file_gen_main.c:158, 340-341, 491-492, 887-888, 1035-1037; dlms_cdf_ls.c:114-157 (read_meter_status)	HSCAN ... 0 MATCH reads only the first page, so meters are "not found" at random

#	Severity	Location	Issue
F9	High	modbus_json_export.c:276-282, 323-340, 1189-1204, 137	NULL dereference and leak in the Modbus nameplate and DCU details
F10	High	file_gen_main.c:940-941, 976-978, 1090-1133	Crash on a missing or non-string field in the meter details JSON
F11	Medium	mqtt_connect.c:1479-1523	TLS: broker hostname is not checked; certificate path handling changed
F12	Medium	mqtt_connect.c:4133-4163, 4060-4078	Retained or duplicate commands are executed again (possible reboot loop)
F13	Medium	get_cfg_request.c:81, 91, 241/256, 385, 575, 709, 851, 1457, 1557, 1677; set_cfg_request.c:174-176	Secrets are returned by get-config and printed in plain text
F14	Medium	mqtt_connect.c:3742-3746, 3896-3897, 2819-2829, 2891-2907, 2968-3011, 3547-3567, 3615-3726, 1717-1768	The server gets no response on most failure paths; large files are sent unframed
F15	Medium	dlms_cdf_ls.c:119-160 (read_meter_status)	Redis reply leaked on every call; reply-owned string freed directly
F16	Medium	dlms_cdf_billing.c:805-830, 879-993; dlms_cdf_mn.c:520, 550-621	No upper bound on billing month range or midnight day count
F17	Medium	file_gen_main.c:848-875, 954-1010, 1146-1219, 1247-1250; health_status.c:966-1125; dlms_cdf_billing.c:394/418; dlms_cdf_event.c:587; dlms_cdf_ls.c:525; dlms_cdf_mn.c:366	Hand-built JSON is not escaped, and failed sections leave it unbalanced
F18	Medium	get_cfg_request.c:1502-1520, 1547-1557, 1657-1677, 237-249	Get-config produces invalid JSON
F19	Medium	set_cfg_request.c:784-830	SERPORT set-config is never applied, and is half-applied on a bad PARITY
F20	Medium	set_cfg_request.c:126-147 and every UPDATE_STR macro	Redis write failures are reported as SUCCESS
F21	Medium	health_status.c:27, 441, 562, 592-618	Meter status matching reads and writes outside the valid entries
F22	Medium	modbus_json_export.c:1104-1106	ReadModbus response is invalid JSON when RTU and TCP devices share a slave ID
F23	Medium	modbus_json_export.c:1277, 1291, 1334, 1336	ReadModbus only searches serial port 1
F24	Medium	mqtt_connect.c:2360-2375	Command arguments depend on the order of keys in the JSON
F25	Medium	mqtt_connect.c:1905-1912, 1953-1958	A failed subscribe is never retried
F26	Medium	dlms_cdf_ls.c:271-274, 303-306, 516	LS interval number is the row index, not the time slot
F27	Medium	file_gen_main.c:2179-2190, 2233	Concatenated output path is a 64-byte buffer and may be uninitialised
F28	Medium	dbg_log.c:89	diag_msgs:MQTT_PROC list grows without limit
L1	Low	main.c:1617; module-wide	Redis connection is never re-established
L2	Low	Makefile:39-43	clean misses src/*.o; no header dependencies
L3	Low	set_cfg_request.c:974-975, 1096, 1172	Restart names with spaces (still open)
L4	Low	get_cfg_request.c vs set_cfg_request.c	Get-config and set-config use different key names
L5	Low	set_cfg_request.c (many handlers)	No range or format checks on set-config values
L6	Low	set_cfg_request.c:582-583	IEC104 MODBUS_IOA_OFFSET / COMMANDS_IOA_OFFSET have no reader
L7	Low	get_cfg_request.c (dcu_sn at 11 places); mqtt_connect.c:2490-2607; cmd_resp.c:189, 257, 292	Leaks and missing NULL checks
L8	Low	health_status.c:966-978, 1127	APPEND overflow handling makes truncation worse
L9	Low	health_status.c:193-198	redis_uptime used uninitialised on Redis errors
L10	Low	health_check.c:34-51	send_hc_msg: unchecked fgets, possible overflow, key from argv[0]
L11	Low	mqtt_connect.c:748-753, 740-744	Commands silently dropped or truncated

#	Severity	Location	Issue
L12	Low	mqtt_connect.c:437-451, 349-366	Race between callback check and client destroy
L13	Low	main.c:989, 1637; mqtt_connect.c:942-965	Other unsynchronised cross-thread access
L14	Low	dlms_cdf_event.c:151-156, 185-190, 225-230; dlms_cdf_ls.c:279-284; dlms_cdf_billing.c:77-82	OD table left behind on error returns
L15	Low	all four dlms_cdf_*.c (e.g. billing.c:486-494, event.c:715-749, ls.c:643-677)	Paths used uninitialised when get_base_path() fails
L16	Low	dlms_cdf_ls.c:193-203	block_int / demand_int_period ignored when stored as numbers
L17	Low	modbus_json_export.c:73, 134-147, 180-197	Modbus JSON escaping and number formatting
L18	Low	file_gen_main.c:1478-1499, 1689-1708	Missing error checks around file I/O
L19	Low	redis_hash_generator.c (whole file)	Unused tool code linked into the daemon
L20	Low	get_cfg_request.c:1250-1274, 1697-1736	modrtu values written raw; no get-config for SERPORT
L21	Low	dlms_cdf_billing.c:446-458; main.c:1451; include/mqtt_module.h	Dead code with bugs

High: 10 Medium: 18 Low: 21

High and Medium findings - details

F1. Shell and pppd option injection through MODEM fields [High]

Location: set_cfg_request.c:288-291; Modem_PPP_IPSEC/ppp_monitor.sh:530-545, 626-651, 682

Problem: USERNAME, PASSWORD, APN and DIAL_NUM are written to Redis without validation. ppp_monitor.sh pastes them unquoted into a generated /etc/ppp/ppp-on (TELEPHONE=\${_phone_num}, PASSWORD=\${_password}), which it then executes as root, and into the chat script and /etc/ppp/options (name \${_username}).

Impact: A set-config with "DIAL_NUM":"*99#;reboot" runs reboot (or any command) as root the next time the PPP link is set up. A newline in USERNAME adds arbitrary pppd options. This is a second injection path, separate from N1 (IPsec).

Suggested fix: In C, whitelist characters per field (APN [A-Za-z0-9.-], DIAL_NUM [0-9*#+], no quotes, \$, :, backticks or control characters in username/password) and reject the request otherwise. In the script, quote the values when writing ppp-on, or load them from a separate file instead of generating shell code.

Verification: Confirmed

F2. Uninitialised file names passed to system("rm ...") and fopen [High]

Location: file_gen_main.c:2103-2106, 2196-2214 (same pattern 2259/2277, 2308/2318, 2345/2353, 2379/2387)

Problem: ls_file_name, mn_file_name, billing_file_name and event_file_name are uninitialised char[128]. Every generator can return -1 before it copies the output name (e.g. dlms_cdf_ls.c returns at 127-263, copies at 625). The return codes are ignored and the code runs sprintf(file_rem_cmd[128], "rm %s", name); system(file_rem_cmd). The generate_mqtt_*_json wrappers likewise fopen() the uninitialised name.

Impact: When a meter's status is missing (common, see F8), root runs rm on leftover stack contents. A name of 125+ characters, or garbage without a NUL, overflows file_rem_cmd. In GetDay loops the stack often still holds the previous day's file name, so a stale file can be sent as the answer with status 0; the server never gets an error.

Suggested fix: Initialise the buffers to "", skip a part when rc != 0 and report the failure, replace system("rm") with unlink(), and pass buffer sizes to the generators.

Verification: Confirmed

F3. Stack buffer overflow from METER_NAME [High]

Location: set_cfg_request.c:847 (MODBUS_TCP), :940 (MODBUS_RTU)

Problem: strcpy(meter_name, item->valuestring) copies the server-supplied METER_NAME into char meter_name[64] with no length check. The DLMS handlers (1053/1133) already use snprintf.

Impact: A set-config with a METER_NAME longer than 63 bytes overwrites the MQTT worker's stack (process runs as root): crash, or possibly code execution.

Suggested fix: Use snprintf(meter_name, sizeof meter_name, ...) and reject names that are too long.

Verification: Confirmed

F4. Get-config DLMS_ETHERNET returns (char*)-1 and crashes [High]

Location: get_cfg_request.c:1626-1630; mqtt_connect.c:3963-3968

Problem: export_dlms_ethernet_cfg() returns char*, but when the hash ethernet_meter_cfg does not exist it does return -1. The caller only checks for NULL and then runs strlen(json) and free(json) on address -1. jb.data and dcu_sn also leak on that path.

Impact: Any get_cfg DLMS_ETHERNET on a DCU with no Ethernet meters causes a SIGSEGV in re_mqtt_proc. Repeating the request keeps the process restarting.

Suggested fix: free(jb.data); free(dcu_sn); return NULL (or return an empty METER_DATA). Build with -Werror=int-conversion so the compiler catches this.

Verification: Confirmed

F5. Logging from Paho threads uses the shared Redis context and log file without locking [High]

Location: dbg_log.c:64-96, 103-136, 291-333; main.c:117-120, 1472; mqtt_connect.c:914, 1059-1067, 1323-1329, 4145

Problem: log_write() calls add_process_diag(), which runs HGET diag_enable every 5 s (and LPUSH per message when diag is on) on the global ctx. log_write() also calls log_rotate(), which fclose(s) and re-fopen(s) g_log_fp with no lock. Paho callback threads log constantly (PAHO TRACE at PROTOCOL level, on_connect_failure, connectionLost, on_message_arrived) while the worker uses the same ctx. localtime() is also shared.

Impact: Two threads use one hiredis context at once: replies get swapped (the worker can read an HGET/LPUSH reply as its own config value) or the context is corrupted and the process crashes. Rotation (every 2 MB, frequent with PAHO TRACE) while another thread is in fprintf is a use-after-free. This undoes the old fix 6 ("callbacks only set flags").

Suggested fix: One mutex around the whole body of log_write. Do not touch Redis from log_write: queue diag text for the worker, or give the logger its own redisContext. Use localtime_r. Set the Paho trace level to ERROR in production.

Verification: Confirmed (call paths read; the crash depends on timing)

F6. Global OD flags stay set and redirect cyclic data to the OD tables [High]

Location: mqtt_connect.c:2798/2855, 2887/2901, 2961/2982, 2990/3005; dlms_cdf_ls.c:244; dlms_cdf_event.c:106, 209-214; dlms_cdf_mn.c:36; dlms_cdf_billing.c:42-50, 196-200; file_gen_main.c:854

Problem: The generators choose the *_od_* SQLite table, and drop it after reading, purely from the globals ls/midnight/event/billing_cmd_redis_resp. The MN, event and billing flags are reset to 0 only on success; the LS flag stays 1 for the whole multi-day FetchDay. json_write_header also uses these flags to label files OD_RESP_MESSAGE. The cyclic generate_profile_json (main.c:1070) and parse_getday_cmd run on the same worker thread between FetchDay responses. In billing, the static od_table is dropped on normal reads too.

Impact: One FetchDay with no data (e.g. billing for an empty month) is enough: from then until restart, cyclic billing/midnight/event files read the OD tables (stale or dropped) and every cyclic file is labelled OD_RESP_MESSAGE. During a multi-day FetchDay LS, cyclic LS files for every meter come out empty, and the cyclic run drops the FetchDay meter's OD table so the next day response finds no data. With all four flags set the event query has no WHERE clause and returns the whole table.

Suggested fix: Pass "use OD table" and the request's transaction as parameters from the FetchDay response handler; never read these globals inside the generators. Drop only the table the current call actually read.

Verification: Confirmed

F7. check_redis_resp has no timeout and puts the FetchDay SEQ_NUM on cyclic messages [High]

Location: mqtt_connect.c:3896-3897, 2838-2860, 3749; file_gen_main.c:863-871

Problem: FetchDay sets check_redis_resp = 1 before generate_redis_list() and ignores its result. If that call fails, or the DLMS poller never answers (meter offline), the flag never clears. While it is 1, json_write_header writes SEQ_NUM = cpy_cmd.transaction instead of the cyclic counter, and cpy_cmd is overwritten by every incoming command.

Impact: Cyclic instantaneous and profile files carry an old FetchDay transaction number, possibly permanently, so the server can match cyclic data to an old request or drop it. OD responses get the wrong SEQ_NUM if any other command arrives in between (extends N9).

Suggested fix: Clear the flag and send a failure response when generate_redis_list fails; add a per-request timeout (e.g. 10 min); pass the transaction ID explicitly instead of using cpy_cmd.

Verification: Confirmed

F8. HSCAN ... 0 MATCH reads only the first page, so meters are "not found" at random [High]

Location: file_gen_main.c:158, 340-341, 491-492, 887-888, 1035-1037; dlms_cdf_ls.c:114-157 (read_meter_status)

Problem: Each lookup sends one HSCAN <hash> 0 MATCH <pattern> and never follows the returned cursor. meter_status and the instantaneous hash hold JSON values longer than 64 bytes, so Redis stores them as a hashtable; HSCAN then returns about 10 slots per call and MATCH filters after that. In read_meter_status the check strstr(field_key, serial) is always true, so the returned key is never checked.

Impact: With more than about 10 fields in the hash, meters whose field is not in the first page get "entry missing": no instantaneous data, no NP/DCU_DETAILS/D1, broken JSON, and LS/MN/event/billing generation (cyclic, GetDay and FetchDay) fails. Which meters fail depends on hash layout and changes as meters are added, so it looks random.

Suggested fix: Build the exact field name and use HGET. If a wildcard is really needed, loop HSCAN until the cursor returns "0" and compare the returned key.

Verification: Confirmed in code; how often it triggers depends on the number of fields in meter_status

F9. NULL dereference and leak in the Modbus nameplate and DCU details [High]

Location: modbus_json_export.c:276-282, 323-340, 1189-1204, 137

Problem: redis_hget() returns NULL when a field is missing, and jbuf_append_escaped() dereferences the string without a NULL check. The 7 strdup'd strings at 276-282 are never freed (dcu_name and dcu_ser are not even used). export_dcu_details has the same problem in the ReadModbus path.

Impact: If any of attribute1-attribute5 is missing (fresh DCU, Redis flushed or not yet configured), re_mqtt_proc segfaults on every Modbus publish cycle and every ReadModbus command, so it keeps restarting. Otherwise it leaks about 7 strings per cycle.

Suggested fix: Read into stack buffers with rget_str (as export_cmd_dcu_nameplate does), or check for NULL and free everything. Make jbuf_append_escaped treat NULL as "".

Verification: Confirmed

F10. Crash on a missing or non-string field in the meter details JSON [High]

Location: file_gen_main.c:940-941, 976-978, 1090-1133

Problem: Every value is read as cJSON_GetObjectItem(j, "x")->valuelisting with no check, then passed to strcmp/printf. A missing key is a NULL dereference; a number gives a NULL valuelisting. ipv4_address is still read at 1117-1118, although the comment at 554-555 says it was dropped from the details JSON.

Impact: Any meter whose details JSON lacks location, port, met_id or one of the 14 D1 keys crashes the daemon on every cycle.

Suggested fix: Add a helper get_str(obj, key) that returns "" unless cJSON_IsString is true, and use it everywhere.

Verification: Crash path confirmed; whether ipv4_address is actually missing is not verified (producer not in this repository)

F11. TLS: broker hostname is not checked; certificate path handling changed [Medium]

Location: mqtt_connect.c:1479-1523

Problem: MQTTAsync_SSLOptions.verify keeps its initializer value 0, so Paho checks the certificate chain but not that it matches broker_ip. insecure (from Redis/set-config) turns certificate checking off completely. The new code passes ca_certificate/client_certificate/client_key from Redis straight to Paho as paths; the previous version (commented out at 1418-1458) prefixed MQTT_1_CERTS_LOC.

Impact: Anyone holding any certificate signed by the trusted CA can impersonate the broker and inject commands such as Reset or set-config. If Redis still stores bare file names, TLS loading fails.

Suggested fix: Set ssl->verify = 1. Rebuild the full path from the certificate directory and accept only plain file names (no / or ..).

Verification: verify: Confirmed. Path regression: Plausible (stored format not seen)

F12. Retained or duplicate commands are executed again (possible reboot loop) [Medium]

Location: mqtt_connect.c:4133-4163, 4060-4078

Problem: on_message_arrived queues every message without checking message->retained or dup, and nothing de-duplicates by SEQ_NUM. The only authorisation is that the payload contains the DCU serial.

Impact: A Reset published with the retain flag (by mistake, or by any client allowed to publish on the topic) is delivered again on every subscribe: the DCU reboots, reconnects, receives it again and reboots again. With QoS 1, set-config can be replayed the same way.

Suggested fix: Drop messages with retained set; keep the last N SEQ_NUMs and ignore repeats.

Verification: Plausible (missing checks confirmed; depends on broker/server behaviour)

F13. Secrets are returned by get-config and printed in plain text [Medium]

Location: get_cfg_request.c:81, 91, 241/256, 385, 575, 709, 851, 1457, 1557, 1677; set_cfg_request.c:174-176

Problem: Get-config replies include the MQTT, modem, FTP and meter passwords, the IPsec PRE_SHARED_KEY and the TLS KEY_PASSWORD. set_mqtt_cfg prints the whole DATA object, including PASSWORD, to stdout; export_modrtu_cfg prints the full reply.

Impact: Anyone who can publish to the command topic and knows the DCU serial can read every credential; the secrets also end up in console logs.

Suggested fix: Mask secrets in replies (e.g. "<set>") and remove the debug printf calls.

Verification: Confirmed

F14. The server gets no response on most failure paths; large files are sent unframed [Medium]

Location: mqtt_connect.c:3742-3746, 3896-3897, 2819-2829, 2891-2907, 2968-3011, 3547-3567, 3615-3726, 1717-1768

Problem: A command that fails to parse is only printed to stderr. FetchDay/GetDay generation failures are only logged. mqtt_send_file returns void: if chunk N fails, chunks 1..N-1 are already published and the caller still removes the file. Files larger than one chunk are sent as raw byte slices with no sequence information.

Impact: The server gets an ACK (sent before validation) and then nothing, or a truncated JSON fragment. It cannot tell "no data" from "failed" or "in progress".

Suggested fix: Return a status from mqtt_send_file and parse_getday_cmd and send failure_resp_msg on every error. Add chunk index/total metadata or limit the file size.

Verification: Confirmed

F15. Redis reply leaked on every call; reply-owned string freed directly [Medium]

Location: dlms_cdf_ls.c:119-160 (read_meter_status)

Problem: freeReplyObject(r) is never called on the success path or the !json_str path. Instead free(json_str) frees r->element[1]->element[i+1]->str, a buffer owned by the reply.

Impact: The reply tree leaks on every generator call (4 per meter per cyclic run, plus one per day in GetDay/FetchDay), so memory grows steadily. Adding freeReplyObject later without removing the free would cause a double free.

Suggested fix: Parse with cJSON_Parse(value), remove free(json_str), and call freeReplyObject(r) on every path.

Verification: Confirmed

F16. No upper bound on billing month range or midnight day count [Medium]

Location: dlms_cdf_billing.c:805-830, 879-993; dlms_cdf_mn.c:520, 550-621

Problem: Billing accepts any MM_YYYY range (e.g. 01_0001..12_9999, about 120,000 iterations). Midnight takes num_days from the request or Redis response and does calloc(num_days, ...). Each iteration opens SQLite and runs a query.

Impact: One GetDay BILLING/MIDNIGHT request blocks the worker for minutes; the heartbeat stops and dcu_mon_proc restarts the process after 180 s (same as N5, which only covers LS).

Suggested fix: Cap the range (e.g. 24 months for billing, 62 days for MN) and reject larger requests; open the DB once per request.

Verification: Confirmed

F17. Hand-built JSON is not escaped, and failed sections leave it unbalanced [Medium]

Location: file_gen_main.c:848-875, 954-1010, 1146-1219, 1247-1250; health_status.c:966-1125; dlms_cdf_billing.c:394/418; dlms_cdf_event.c:587; dlms_cdf_ls.c:525; dlms_cdf_mn.c:366

Problem: Values from Redis and SQLite (attributes, DCU location, meter location/BAY, ISP operator, OBIS text values) are written with "%s" and no escaping; a NULL from redis_hget prints as "(null)". When json_write_general or json_write_d1 return early, the header has opened { but the footer closes two levels.

Impact: A " or \ in any of these values makes the file invalid JSON: concatenate_files drops that meter's whole profile every cycle, INST is published invalid with status 0, and every health message is rejected until the value is changed.

Suggested fix: Build these messages with cJSON (already linked), or escape every string and write placeholders/fail on error paths.

Verification: Confirmed (file_gen, health); Plausible for the DLMS value fields

F18. Get-config produces invalid JSON [Medium]

Location: get_cfg_request.c:1502-1520, 1547-1557, 1657-1677, 237-249

Problem: DLMS_SERIAL writes "SERIAL_PORT_n":{ before checking the hash exists and then continues without closing it. When an ADD_STR field is missing the following "," is still written (".,," or ",,,"). enable_meter defaults to -1, so a missing field is reported as YES. MODEM SIM1/SIM2 end with "," when phone_numN is not set.

Impact: The server cannot parse the reply but still sees CMD_STATUS 0.

Suggested fix: Build replies with cJSON, or write separators only between fields that were actually written.

Verification: Confirmed

F19. SERPORT set-config is never applied, and is half-applied on a bad PARITY [Medium]

Location: set_cfg_request.c:784-830

Problem: Unlike every other handler there is no SADD proc_restart; modrtu_master and SerDaProc load port settings only at startup. A bad PARITY returns -1 after baud rate, data bits and stop bits were already written.

Impact: The server gets SUCCESS but nothing changes until a reboot; after a reboot a half-written configuration takes effect even though the request was reported as failed.

Suggested fix: Validate all fields first, then request a restart of modrtu_master_<n> or SerDaProc_<n> depending on device_type.

Verification: Confirmed

F20. Redis write failures are reported as SUCCESS [Medium]

Location: set_cfg_request.c:126-147 and every UPDATE_STR macro

Problem: A NULL or REDIS_REPLY_ERROR reply from HSET is ignored; hset_int_field returns 1 either way and set_* returns 0. (Different from N14, which is about invalid input values.)

Impact: After a lost Redis connection every set-config replies SUCCESS without writing anything.

Suggested fix: Check the reply type and ctx->err and return -1; ideally wrap the writes in MULTI/EXEC.

Verification: Confirmed

F21. Meter status matching reads and writes outside the valid entries [Medium]

Location: health_status.c:27, 441, 562, 592-618

Problem: The match loops use `j <= count` and write `meters[j]`; with `MAX_METERS` redefined to 64, `count == 64` writes `meters[64]`. The serial loop runs `i <= count` and writes `meters[i-1]`, so id 0 writes `meters[-1]`. `meters[count]` is never cleared between the eth and serial passes.

Impact: Memory around the static meters array is corrupted, and a serial meter without a meter_status entry is reported with another meter's COMM_STATUS.

Suggested fix: Store the Redis meter index in meter_entry_t, match on index plus port, and keep every index below count.

Verification: Plausible (index arithmetic confirmed; depends on the ids the poller writes)

F22. ReadModbus response is invalid JSON when RTU and TCP devices share a slave ID [Medium]

Location: modbus_json_export.c:1104-1106

Problem: export_cmd_modrtu_device and export_cmd_modtcp_device each append {...} with no comma between them.

Impact: With slave_id 1 on both an RTU and a TCP device (common), the response is "MODBUS_DEVICES":[...]{...}], which the server rejects.

Suggested fix: Append "," between the two when both export a device, or use one list writer.

Verification: Confirmed

F23. ReadModbus only searches serial port 1 [Medium]

Location: modbus_json_export.c:1277, 1291, 1334, 1336

Problem: The keys are hard-coded as `modrtu_serial1_...` and `modrtu:1:...`, while the cyclic export iterates ports 0..1. `num_points` is not clamped.

Impact: RTU devices on the other port are never found; the command returns an empty or TCP-only result.

Suggested fix: Loop over the ports whose `serial_port_%u_cfg device_type` is Modbus RTU, as the cyclic path does; clamp `num_points` to `MAX_REGS_PER_DEV`.

Verification: Plausible

F24. Command arguments depend on the order of keys in the JSON [Medium]

Location: mqtt_connect.c:2360-2375

Problem: `args[]` is filled in JSON key order and only string values are kept; a numeric value is skipped, so every later index shifts. Downstream code uses fixed indexes (`args[0]` DCU serial, `[1]` meter, `[2]/[3]` dates, `[4]` event category).

Impact: If the server reorders keys or sends a number (e.g. event type 3 instead of "3"), the request is rejected or runs with the wrong dates or category.

Suggested fix: Look each field up by name and accept both numbers and strings.

Verification: Plausible (parser behaviour confirmed; server payload format not checked)

F25. A failed subscribe is never retried [Medium]

Location: mqtt_connect.c:1905-1912, 1953-1958

Problem: When SUBACK reports failure or MQTTAsync_subscribe returns an error, the code only logs it and the broker stays CONNECTED.

Impact: The broker keeps receiving cyclic data but the DCU never receives commands until the link happens to drop, which could be weeks.

Suggested fix: On subscribe failure, move the broker to CLOSING so it reconnects, or retry the subscribe.

Verification: Confirmed

F26. LS interval number is the row index, not the time slot [Medium]

Location: dlms_cdf_ls.c:271-274, 303-306, 516

Problem: interval_num = interval_idx counts returned rows, so after a missing interval every later block gets a lower number. The day window LIKE 'YYYY-MM-DD%' includes 00:00 of the day and excludes 00:00 of the next day. Rows beyond 288 are dropped silently.

Impact: The server may map values to the wrong 15/30-minute slot after a power failure; if meters timestamp blocks at interval end, each day file holds the previous day's last block and misses its own.

Suggested fix: Compute interval_num from the timestamp ((HH*60+MM)/block_int, +1 if 1-based). Confirm the timestamp convention with the poller and adjust the window.

Verification: Plausible (code confirmed; depends on how the server uses the number)

F27. Concatenated output path is a 64-byte buffer and may be uninitialised [Medium]

Location: file_gen_main.c:2179-2190, 2233

Problem: output_file_name[64] receives "%s/data/%s_%s" with base_path (up to 255 bytes), date and serial. If get_base_path fails the buffer is used uninitialised by fopen(..., "w").

Impact: With a base path longer than about 35 characters the serial is cut off and different meters share one file name, so one meter's profile can be published as another's.

Suggested fix: Use a PATH_MAX buffer, check the snprintf result, and return when get_base_path fails.

Verification: Plausible (base path value not in the repository)

F28. diag_msgs:MQTT_PROC list grows without limit [Medium]

Location: dbg_log.c:89

Problem: LPUSH with no LTRIM (dcu_mon_proc's logger trims its own list). While diag is on, every log line is pushed.

Impact: Redis memory grows by roughly 100 MB per day if nothing drains the list, which can lead to OOM and affect every process using Redis.

Suggested fix: LTRIM diag_msgs:MQTT_PROC 0 N-1 after each push.

Verification: Plausible (no consumer of the key found in the repository)

Low findings - details***L1. Redis connection is never re-established [Low]***

Location: main.c:1617; module-wide

ctx is created once, there is no redisReconnect and ctx->err is not checked. After a Redis restart every command fails and dcu_mon_proc kills the process after 180 s. Reconnect with backoff in the worker loop.

Verification: Confirmed

L2. clean misses src/*.o; no header dependencies [Low]**Location:** Makefile:39-43

clean deletes *.o in the module root but objects are in src/ (hence the untracked src/*.o files). Objects do not depend on headers, so a change to general.h (e.g. PAYLOAD_BUFFER_SIZE, cmd_request_t) can leave mixed struct layouts in the binary. Add -MMD -MP, build into build/, ignore *.o in git.

Verification: Confirmed***L3. Restart names with spaces (still open) [Low]*****Location:** set_cfg_request.c:974-975, 1096, 1172

"modrtu_master 0", "SerDaProc 1", "ethDaProc %d" are pushed to proc_restart, but dcu_mon_proc expects underscores, so these restarts never happen. Already reported in the dcu_mon_proc review of 30 Sep; the fix belongs in this module.

Verification: Confirmed***L4. Get-config and set-config use different key names [Low]*****Location:** get_cfg_request.c vs set_cfg_request.c

e.g. IPsec IKE_LIFETIME/IKELIFETIME, PHASE1_DH_GROUP/PHASE1_DH, MOBIKE/MOBIK_MODE, AGGRESSIVE_MODE/AGGR_MODE, CLOSE_ACTION/CLOSEACTION; IEC ASDU_ADDR/ASDU_ADDRESS, MASTER_0_IP/MASTER0_IP, CYCLIC_INTERVAL/CYCLIC_INT, IOA_OFFSET/DLMS_IOA_OFFSET, LINK_ADDR/LINK_ADDRESS; MQTT BROKER_IP_URL/BROKER_IP, HC_PUB_INTERVAL/HEALTH_INTERVAL; Modbus RESPONSE_TIMEOUT/RESP_TIMEOUT, POLL_FAULTY_COUNT/POLL_SKIP_COUNT; typo MTER_IP_ADDRESS (get_cfg 1657). Values the server edits and sends back are silently ignored. Use one shared key table.

Verification: Confirmed in code (spec document not checked)***L5. No range or format checks on set-config values [Low]*****Location:** set_cfg_request.c (many handlers)

Consumers truncate silently: slave_id is cast to uint8_t (300 becomes 44), ports to uint16_t (70000 becomes 4464). IPs, baud rate, data/stop bits and certificate paths are free text. Add per-field validators.

Verification: Confirmed***L6. IEC104 MODBUS_IOA_OFFSET / COMMANDS_IOA_OFFSET have no reader [Low]*****Location:** set_cfg_request.c:582-583

iec104 reads only ioa_offset, so the server is told the offsets changed but IEC104 IOAs do not change. Implement or reject.

Verification: Plausible***L7. Leaks and missing NULL checks [Low]*****Location:** get_cfg_request.c (dcu_sn at 11 places); mqtt_connect.c:2490-2607; cmd_resp.c:189, 257, 292

dcu_sn from redis_hget is never freed in get-config and is written unescaped ("null" if missing). generate_redis_list has no return value on success, leaks root/json_str/data on errors and passes a possible NULL from cJSON_Print to redisCommand. processServerMsg has return; without a value in an int function. cmd_resp.c strcpy's cJSON_PrintUnformatted without a NULL check.

Verification: Confirmed***L8. APPEND overflow handling makes truncation worse [Low]*****Location:** health_status.c:966-978, 1127

On truncation written still grows past out_sz, so the next sprintf gets a wrapped size and writes past the 128 KB stack buffer, and the function returns 0. Latent today (about 63 KB for 190 meters). Clamp and return -1.

Verification: Confirmed

L9. redis_uptime used uninitialised on Redis errors [Low]

Location: health_status.c:193-198

redis_hget clears the buffer only for NIL replies; on NULL/ERROR atoi reads stack garbage, so ACTIVE_SINCE is garbage. Initialise to {0}.

Verification: Confirmed

L10. send_hc_msg: unchecked fgets, possible overflow, key from argv[0] [Low]

Location: health_check.c:34-51

sprintf into temp_buff[256] can write 267 bytes; if the binary is renamed or started through a symlink, the heartbeat key no longer matches re_mqtt_proc_hc_up_time and dcu_mon restarts the process every 180 s. Hard-code the key, use snprintf.

Verification: Confirmed

L11. Commands silently dropped or truncated [Low]

Location: mqtt_connect.c:748-753, 740-744

When the 8-slot queue is full the oldest command is discarded; payloads of 4096 bytes or more are cut to 4095 bytes (invalid JSON). No response is sent in either case. Reply busy/failure.

Verification: Confirmed

L12. Race between callback check and client destroy [Low]

Location: mqtt_connect.c:437-451, 349-366

in_callback is checked under the lock, but the lock is released before brk_destroy_client(), and the client is destroyed without MQTTAsync_disconnect. A callback can start in that gap (rare crash).

Verification: Plausible

L13. Other unsynchronised cross-thread access [Low]

Location: main.c:989, 1637; mqtt_connect.c:942-965

dcu_netlog polling runs from both the main and worker threads (dcu_netlog.c not in repo). on_connect_success writes last_mqtt*/current_active_mqtt* from the Paho thread without a lock (mostly harmless).

Verification: Plausible / Confirmed

L14. OD table left behind on error returns [Low]

Location: dlms_cdf_event.c:151-156, 185-190, 225-230; dlms_cdf_ls.c:279-284; dlms_cdf_billing.c:77-82

Same as N13 (midnight) in the event, LS and billing readers: early returns skip drop_table, so leftover OD rows merge into the next on-demand response.

Verification: Confirmed

L15. Paths used uninitialised when get_base_path() fails [Low]

Location: all four dlms_cdf_*.c (e.g. billing.c:486-494, event.c:715-749, ls.c:643-677)

sqlite3_open and fopen still run with stack garbage. Return -1, as generate_midnight_json already does.

Verification: Confirmed

L16. block_int / demand_int period ignored when stored as numbers [Low]

Location: dlms_cdf_ls.c:193-203

Only cJSON strings are parsed, so a numeric value becomes BLOCK_INTERVAL 0. Accept cJSON_IsNumber too.

Verification: Plausible

L17. Modbus JSON escaping and number formatting [Low]**Location:** modbus_json_export.c:73, 134-147, 180-197

Control bytes below 0x20 are not escaped; a failed realloc silently drops fragments. Values without '.' go through atol, so "1e-05" becomes 1 and nan/inf become 0. Use \u escapes, an error flag, and strtod.

Verification: Confirmed / Plausible**L18. Missing error checks around file I/O [Low]****Location:** file_gen_main.c:1478-1499, 1689-1708

ftell -1 leads to malloc(0) and a write to buf[-1]; fread result unchecked; NULL from cJSON_Print passed to fputs.

Verification: Confirmed**L19. Unused tool code linked into the daemon [Low]****Location:** redis_hash_generator.c (whole file)

The Makefile wildcard links it into re_mqtt_proc though nothing calls it. It exports generic global names (log_message, get_timestamp, init_logging, ...) and has unbounded strcpy/strcat. Remove it from the build.

Verification: Confirmed**L20. modrtu values written raw; no get-config for SERPORT [Low]****Location:** get_cfg_request.c:1250-1274, 1697-1736

scale_fact and tol_* go in with "%s" (a value like "1,5" breaks the JSON); get-config returns NULL for SERPORT although set-config supports it.

Verification: Plausible / Confirmed**L21. Dead code with bugs [Low]****Location:** dlms_cdf_billing.c:446-458; main.c:1451; include/mqtt_module.h

generate_billing_cdf reads uninitialised date (should be month); load_mqtt_ssl_cfg dereferences r before the NULL check; mqtt_module.h declares a conflicting mqtt_cfg_t. None are reachable today; delete them.

Verification: Confirmed**Recommended order of work**

1. Close the injection and remote-crash paths: F1 and N1 (validate every set-config string that reaches a script; stop generating shell code from Redis values), N2, F3, F4, F9, F10.
2. Restore the broker field names (old findings 1-3) so MQTT set-config and get-config work at all.
3. Make logging thread-safe and stop using Redis from log_write (F5); add Redis reconnect (L1).
4. Remove the global on-demand state: pass use_od / transaction / dates as parameters (F6, F7, N9, old 10); initialise file names and handle generator failures (F2); replace HSCAN lookups with HGET (F8); fix the reply leak (F15).
5. Send a response for every failure, cap all date ranges, keep the heartbeat running (F14, F16, N5).
6. TLS and command handling: verify the hostname, drop retained/duplicate commands, retry subscribe (F11, F12, F25); mask secrets in get-config (F13).
7. Get-config/set-config correctness: valid JSON, SERPORT, Redis errors, shared key table, restart names (F18-F20, L3-L5).
8. Remaining Medium and Low findings; build with -Wall -Werror=implicit-function-declaration -Werror=int-conversion and fix the Makefile (L2) so the compiler catches this class of bug.